Fraud Feature Engineering, Explained:
Features, Results, and Next Steps

Eric, Jean, Salamot, Salvador, Susie
Banking 1B · Break Through Tech × Mastercard

October 26, 2026

1 Overview

Feature engineering turns the raw IEEE-CIS data into two model-ready tables: 537 columns for tree models (XGBoost, LightGBM) and 515 normalized columns for a neural net. No model is trained here. Every rule is learned from the training period only and applied unchanged to later periods.

- **Starting data:** the raw Kaggle files, loaded and joined exactly as in the EDA. The training period is the same 472,432 transactions the EDA analyzed.
- **Inputs:** the EDA decision log, the challenge advisor's guidance (Debasmita), and six teammate notebooks (Eric, Jean, Salvador, Jennifer, Susie, Sal).
- **Outputs:** saved tables for train, validation, holdout and Kaggle test, plus labels, metadata, the config and every fitted rule. The modeling pipeline loads these directly.

Advisor guidance	How it is handled
Avoid one-hot for columns with many categories	Frequency encoding for the 14 many-category columns
Mean encoding only works if fraud rates stay stable	Jennifer's out-of-time target encoding for three stable columns, with a stability table
Trees need derived features; neural nets need normalization	A tree table and a separate normalized neural-net table
Start with all features, then reduce within 1 to 2%	Named feature sets, including a reduced-V set
Judge models by detection rate and TFPR with a KS table	Reusable KS table function, demoed on one feature
Keep a config per run	Config saved with every output

2 Data and split

The labeled data is split by time into 80% training, 10% validation and 10% holdout. Training is identical to the EDA's; the EDA's 20% validation period is cut in half to create a labeled holdout, the out-of-time set the advisor asked for.

Period	Rows	Days	Fraud rate	Used for
Train	472,432	1 to 141	3.51%	Learning every rule and fitting models
Validation	59,054	141 to 161	3.13%	Comparing feature sets and settings
Holdout	59,054	161 to 182	3.75%	One final, honest score
Kaggle test	506,691	213 to 395	no labels	Kaggle submission only

- **Why a holdout when Kaggle test exists:** Kaggle test has no labels, so it cannot produce detection rate, TFPR or a KS table. Validation gets reused across many experiments and drifts optimistic; the holdout is scored once.
- **No overlap:** each boundary moves to the start of a timestamp, so payments made in the same second never land in two periods.
- **Fraud shifts over time:** 3.13% vs 3.75% in three weeks. Precision and PR-AUC move with the fraud rate, so compare periods carefully.

3 Categories and frequency encoding

Columns with 10 or fewer categories are one-hot encoded for the neural net; the 14 with more get frequency encoding. One-hot encoding those 14 would add about 14,400 mostly-zero columns (12,730 from `card1` alone); frequency encoding adds 14.

Route	Columns	Tree table	Neural-net table
Few (10 or fewer)	ProductCD, card4, card6, DeviceType, M4	categories	one-hot
Many	card1, card2, card3, card5, addr1, addr2, both email domains, P_provider, device_family, id_14, id_30, id_31, id_33	raw plus `freq_`	`freq_` only

Frequency encoding replaces each category with its share of training transactions. Missing values get their own share; a value never seen in training gets 0. No fraud labels are used, so there is no leakage risk.

$$\text{freq}(c,k) = \frac{\text{training transactions with } c = k}{\text{all training transactions}}$$

Example: if 38% of training transactions use gmail.com, every gmail.com transaction gets 0.38 in `freq_P_emaildomain`, in any period.

Why it helps: raw card codes are labels, not quantities (13926 is not "more" than 2755). Frequency gives the model a meaningful order, from rare to common, and lets it learn patterns about groups of rare values it could never learn one by one.

`id_14` **(from Salvador):** a timezone offset stored as a number. It is treated as a category for the neural net (frequency only) and kept as a number in the tree table, where neighboring offsets are neighboring regions.

Mean encoding is added for the stable columns. The advisor suggested it for many-category columns but warned it only works if fraud rates stay stable over time. Jennifer's target encoding (Section 6) covers `card2` and both email domains; `addr1` is left out because its fraud ranking reshuffled over time (rank correlation 0.38).

4 Amount, time and email features

The most important new amount feature compares each payment with its product's typical amount, because products differ widely: product R's training median is 125, product S's is 30.

$$\text{amount_to_product_median} = \frac{\text{amount}}{\max(\text{training median for this product},\ 1)}$$

A 150 payment is 3.0 times typical on product H (median 50) but only 1.2 times typical on product R (median 125).

Feature	What it captures	Source
`amount_to_product_median`	Amount relative to its product's typical amount	Eric; EDA decision
`amount_train_frequency`	How common the exact amount is in training	Eric; EDA decision
`amount_fractional_unit`	The part after the decimal point	Eric
`amount_has_subcent_precision`	Amount recorded beyond whole cents, often a currency conversion	Eric and Salvador
`amount_log1p`	$\log(1 + \text{amount})$, squeezing very large amounts	Eric
`hour_sin`, `hour_cos`	Hour of day on a circle, so 23:00 sits next to 00:00	EDA decision
`weekday_sin`, `weekday_cos`	Day of the 7-day cycle on a circle	Eric and Salvador
`email_domain_relationship`	Purchaser vs recipient domain: same, different, or which is missing	Eric
`has_identity`	Transaction has identity data	EDA decision

Why two coordinates for time: one alone is ambiguous (hour 1 and hour 11 share a sine of 0.26); sine and cosine together place each hour on the circle, like latitude and longitude.

Email relationship, training counts: recipient missing 292,877 (62%), same domain 85,848 (18%), both missing 65,935 (14%), different domains 20,121 (4%), purchaser missing 7,651 (2%). It mostly repeats product information (product W never has a recipient email), so it is low priority.

5 Account history

A second account key gives account history to 58% of address-missing transactions, the group that is 11.4% of training but 37% of all fraud. Before it, that group had no history at all.

Why account keys: fraud often shows up as a change in a customer's own behavior, but the data has no customer ID. Each key combines columns that stay the same for one customer.

Key	Built from	Covers
Key 1 (`k1_`)	card1, addr1, D1n (estimated card start day)	The EDA's key; needs an address
Key 2 (`k2_`)	card1, P_emaildomain, D1n	Works without an address; broader, since many people share gmail.com

Rule for keys: never put in a key what you want to watch change. Device and email are counted as history

features instead, so a new device on a known account shows up as a change rather than a new account.

Eight features per key, all from strictly earlier timestamps, never using fraud labels. Worked example for one account:

	Time	Amount	Prior count	Prior avg.	× avg.	Secs since last	Prior emails	Prior devices
T1	1,000	50	0	missing	missing	missing	0	0
T2	5,000	40	1	50	0.80	4,000	1	1
T3	9,000	200	2	45	4.44	4,000	1	1
T4	9,000	60	2	45	1.33	4,000	1	1
T5	20,000	500	4	87.50	5.71	11,000	2	2

T5 is the pattern these features catch: almost six times the usual amount, from a new email and device. T3 and T4 share a timestamp, so neither sees the other. The other two features are `key_missing` and `has_prior`.

Coverage in training:

Group	Key 1 built	Key 2 built	Key 1 history	Key 2 history	Fraud rate
Address missing	0%	95.7%	0%	58.3%	11.4%
Address present	99.8%	82.8%	59.9%	49.3%	2.5%

Within the address-missing group, fraud is 11.9% with Key 2 history and 10.7% without. That yes/no flag is the bluntest feature; the amount and device changes should matter more.

6 Target encoding and account behavior (from Jennifer)

Target encoding of `card2` is now the strongest single feature, at 0.728 validation AUC. Jennifer's block adds 35 columns: 3 target encodings and 32 account-behavior features built on both keys, including Susie's 7-day and 30-day velocity windows.

Target (mean) encoding replaces each category with its smoothed training fraud rate, with smoothing strength 20 pulling rare categories toward the overall rate $\bar{y}$:

$$\text{te}(c,k) = \frac{\text{fraud}(c,k) + 20 \times \bar{y}}{n(c,k) + 20}$$

Example: a card with one transaction that was fraud gets $(1 + 20 \times 0.035)/21 = 8.1\%$, not 100%. A card with 1,000 transactions and 50 frauds gets 4.97%, close to its raw 5%.

- **No leakage:** training is cut into 5 time folds, and each fold is encoded only from earlier folds. Validation, holdout and Kaggle test use rates from all of training.
- **Stability check:** category fraud rates in the first half of training are compared with the second half.

Encoded column	Stability (rank correlation, half 1 vs half 2)	Validation AUC alone
`te_card2`	0.48	0.728
`te_R_emaildomain`	0.68	0.683
`te_P_emaildomain`	0.31	0.589

`te_P_emaildomain` is the weakest on both counts; whether to keep it is an open question.

Account behavior features, from strictly earlier timestamps on Key 1 and Key 2:

Family	What it captures	Best validation AUC
New value	This device, email or address was never used by the account before	0.581
Velocity	Payments and money in the previous hour, day, week and month (week and month from Susie)	0.601
Amount deviation	How unusual the amount is for this account, including a new record high	0.585
Repeat amount	The same exact amount paid before, as in card testing or subscriptions	0.612

A new feature set, `all_no_jennifer`, leaves this block out so the modeling pipeline can test whether it helps.

7 Missingness features

Missing data carries fraud signal, so it is described three ways: which blocks are missing, how much is missing, and the overall pattern. `raw_missing_count`, from this group, is one of the strongest single features (0.678 validation AUC).

Block flags. 67 missingness groups in training: 24 blocks of columns that always go missing together (371 columns), plus 43 single columns. This matches the EDA exactly. Each block gets one 0/1 flag, `na_block_00` to `na_block_23`; the largest block's flag stands in for 46 identical ones.

Counts (from Eric). `raw_missing_count`, `identity_missing_count`, `card_address_missing_count` and `v_missing_count` count missing values per transaction.

Profile clusters (from Jean). k-means with $k = 3$ groups transactions by their 67-digit missingness pattern, fit on training only. The result reproduces the EDA's clusters almost exactly:

Cluster	Training transactions	Fraud rate	EDA
0	354,147	2.13%	2.1%
1	68,414	4.44%	4.4%
2	49,871	12.05%	12.0%

Cluster numbers are labels, not amounts, so `na_cluster` is treated as a category. Tree models see missing values directly; for the neural net, missing values are filled with medians, so these flags are the only record that a value was missing.

8 V-column reduction

129 of the 339 V columns are near-copies of others, leaving 210. This reproduces the EDA's result exactly.

1. Within each missingness block, on a 100,000-row training sample, sort the V columns from most distinct values to fewest.
2. Keep the first.

3. Keep each next column only if its Spearman correlation with every kept column is at most 0.95 in absolute value.

Spearman correlation compares the columns' rankings, so it catches any consistent rising-together relationship, not just straight lines. It checks pairs only; a column that is a combination of several others would need VIF.

How it is used: the neural-net table uses the 210 by default. The tree table keeps all 339, because trees handle near-copies well and the rare transactions where two columns disagree may be the fraud. The reduced version is saved as the feature set `all_reduced_v`, to be kept only if performance drops by no more than 1 to 2%, as the advisor suggested.

9 The two output tables

The advisor's point drives the split: a tree splits on values and ignores scale and outliers, but a neural net does math with the values and needs every input numeric, complete and on a similar scale.

	Tree table	**Neural-net table**
Columns	537	515
Original numbers	kept as is	log1p for skewed columns, clipped at the 0.5th and 99.5th percentiles, standardized
Missing values	left missing (trees handle them)	filled with the training median, plus a 0/1 flag per block and single column
Few-category columns	categories	one-hot
Many-category columns	raw column plus `freq_`	`freq_` only
V columns	all 339	the 210 kept

Standardizing centers each column on its training mean:

$$z = \frac{x - \text{training mean}}{\text{training standard deviation}}$$

Example: a 500 payment becomes $\log(501) \approx 6.22$; with a training mean of 4.2 and standard deviation of 1.0, it becomes about 2.0, two standard deviations above typical.

Category lists are fixed from training, with `__MISSING__` for blanks and `__UNSEEN__` for values never seen in training, so a category has the same code in every period.

Feature sets for comparison, with their sizes from the full run:

Feature set	**Columns**	**What it is**
`raw`	430	Original columns only
`all_no_history`	489	Clean: everything except account history
`all`	537	History-enriched: everything
`all_no_jennifer`	502	Everything except the target encoding and behavior block
`all_reduced_v`	408	Everything except the 129 near-duplicate V columns

These match the team's plan of raw, clean and history-enriched datasets, plus the advisor's reduction test. The neural-net rules are a starting point; Kyaw's research on neural-net requirements may change them, for

example learned embeddings for the many-category columns.

10 Signal check and KS table

No single feature reaches the advisor's target alone. The best, `te_card2`, scores 0.728 ROC-AUC and catches 52% of fraud at 11.3 legitimate transactions flagged per fraud. That is the baseline a real model must clearly beat.

Signal check. Each new feature is used alone as a fraud score, with no model, and scored by ROC-AUC: the chance that a random fraud ranks above a random legitimate transaction. A confusion matrix needs only predictions and labels, so a feature's own values can stand in for a model's score.

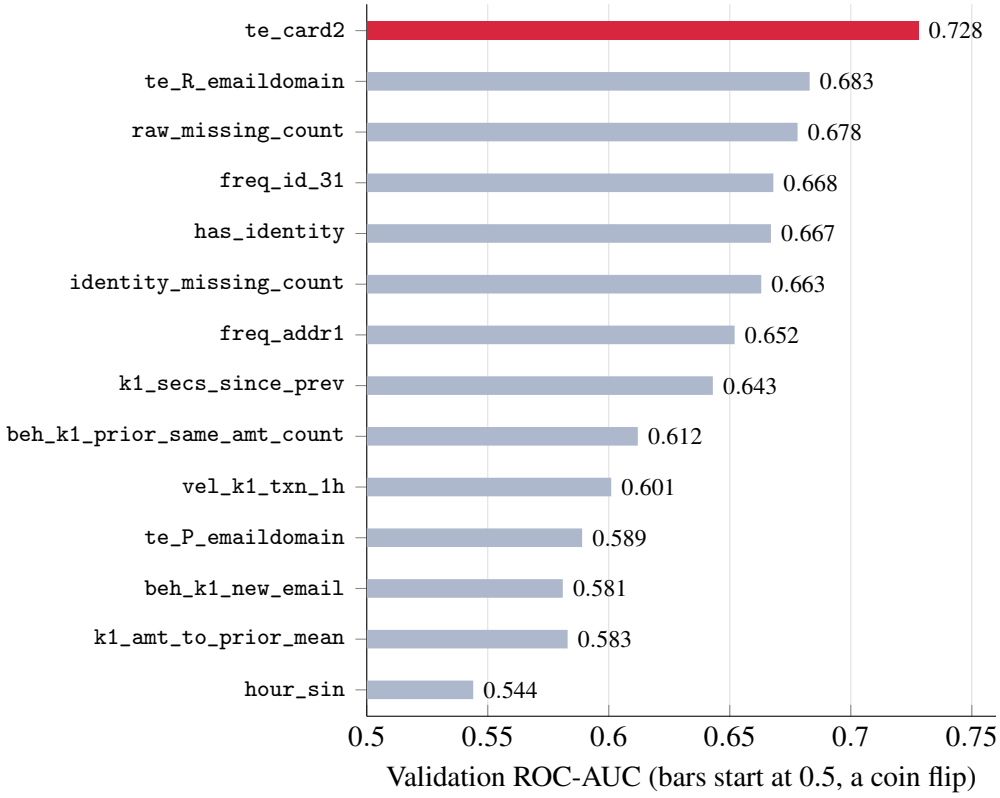


Figure 1: Target encoding of `card2` is the strongest single feature. Selected features, validation period.

The top features mostly repeat one signal: whether a transaction has identity data (7.55% fraud with it vs 2.13% without, per the EDA). Low scores do not mean useless: AUC misses bumps such as the fraud peak in relative hours 5 to 10, and features can help in combination. Key 2 features score low overall because they target the address-missing 11% of transactions.

KS table. The advisor's way to judge a model: sort transactions by score, cut them into 10 equal buckets, and keep running totals of fraud caught and legitimate transactions flagged. KS stands for Kolmogorov-Smirnov, a test of how different two groups of numbers are.

$$\text{TFPR}_b = \frac{\text{legit flagged in buckets } 1 \ldots b}{\text{fraud caught in buckets } 1 \ldots b} \qquad \text{KS}_b = \text{detection rate}_b - \text{share of legit flagged}_b$$

Demo with `te_card2` alone on validation (1,851 fraud, 57,203 legitimate), from the full run:

Flag the top	Fraud caught	Detection rate	Legit flagged	TFPR	KS
10%	742	40.1%	5,164	7.0:1	0.311
20%	958	51.8%	10,853	11.3:1	0.328
30%	1,112	60.1%	16,604	14.9:1	0.310

Compared with `raw_missing_count`, the previous best, encoded `card2` is sharper at the very top (40% of fraud in the top 10%, vs 35% at 8.1:1) but about the same by the top 20% (52% at 11.2:1).

TFPR is a fraud-industry term, the transaction false positive rate: legitimate transactions flagged per fraud caught. Despite the name it is a ratio, as in an eFalcon fraud-scoring patent that defines a TFPR of 10:1 as ten legitimate orders flagged per fraud detected (source). The advisor's target is likely 10:1 at 50 to 60% detection; the notes say "10%". Real models should use 20 or 100 buckets, since fraud teams usually flag only a few percent of transactions.

11 Contributions and open questions

The final notebook combines six teammates' work on top of the EDA.

Teammate	Contributions in the final notebook
Eric	80/10/10 time split; email normalization and providers; frequency encoding; amount, weekday and email relationship features; Key 1 account history; missingness counts; category lists with missing and unseen values; leakage checks; saved outputs
Jean	Device rules from the EDA; missingness profile clusters
Jennifer	Out-of-time target (mean) encoding with a stability table; new device, email and address flags; 1-hour and 24-hour velocity; account amount spread, z-score and record-high ratio; repeated amounts
Susie	7-day and 30-day velocity windows; her amount, time, missingness, email, device and account-history features match Eric's versions already in the notebook
Salvador	`id_14` timezone as a category; three-decimal amount flag; ideas from Kaggle discussions, several already covered
Sal	Second account key; distinct-value history; block flags; reduced-V feature set; neural-net table; signal check; KS table; per-run config; combining the notebooks

For Debasmita

1. Did the target mean a **10:1 TFPR** at 50 to 60% detection? The notes say "10%".
2. What share of transactions does a fraud team typically flag? This sets how finely to cut the KS table.
3. Should we also report the share of fraud dollars caught?
4. Does Mastercard use a standard stability check (such as PSI) before target encoding, and should the final model refit the encodings on train plus validation?

For the team

1. Card and address codes as numbers with frequency encodings, or as categories? Test both.
2. Does Key 2 over-merge people on shared domains like gmail.com? Score it within the address-missing group, then compare models with and without the `k2_` features.
3. Keep one of the repeated identity features (`has_identity`, `freq_id_31`, several block flags) when reducing features?
4. Drop the near-empty block flags `na_block_01`, `02` and `12`?
5. Is a cyclical version of `D9` (from Salvador's D9 $\times$ 24 idea) worth testing?

From Jennifer's block

1. Keep `te_P_emaildomain`? It has the weakest stability (0.31) and a modest 0.589 AUC.
2. `beh_k1_new_email` and `k1_prior_distinct_P_emaildomain` come from the same history and may repeat each other.
3. Add a shorter velocity window, such as 10 minutes?

Next step: one person builds the modeling pipeline end to end and compares the feature sets on validation, reporting AUC, PR-AUC and a KS table.